

HOVERCRAFT: A SCALABLE RAFT MODIFICATION – TLA+ DESIGN AND LIVENESS CONSIDERATIONS

Presented by Parsa VARES – 16.06.2025

Dr. Ovidiu-Cristian MARCU & Dr. Georgios KAFANAS

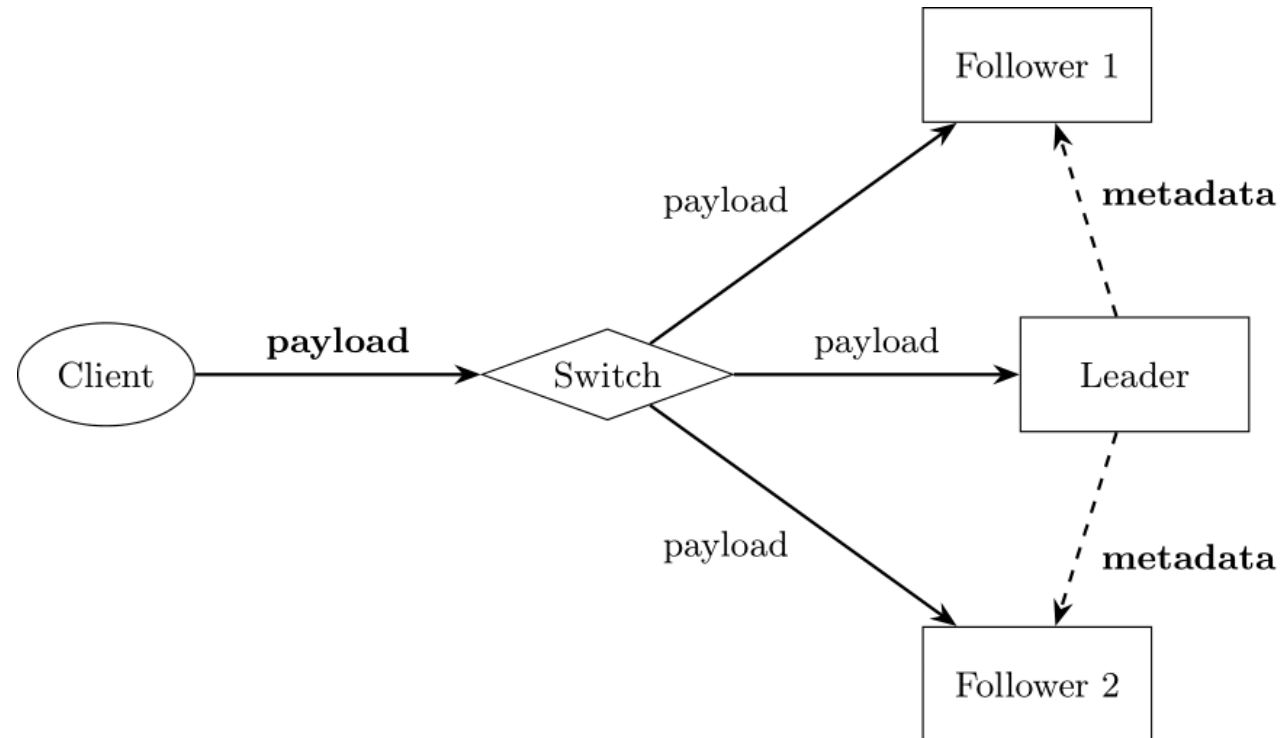




CONTENT:

- ❖ Hovercraft Design – The Switch & Payload Delivery
- ❖ Hovercraft Design – Leader Ordering & Follower Reconstruction
- ❖ Liveness in Hovercraft – Fundamentals & Key Stages
- ❖ Liveness: Replication, Reconstruction & Recovery
- ❖ Overall Liveness Discussion & Project Conclusion

❖ HOVERCRAFT CORE DESIGN: DECOUPLING PAYLOAD & ORDER



Key idea: Separate large payload dissemination from consensus ordering

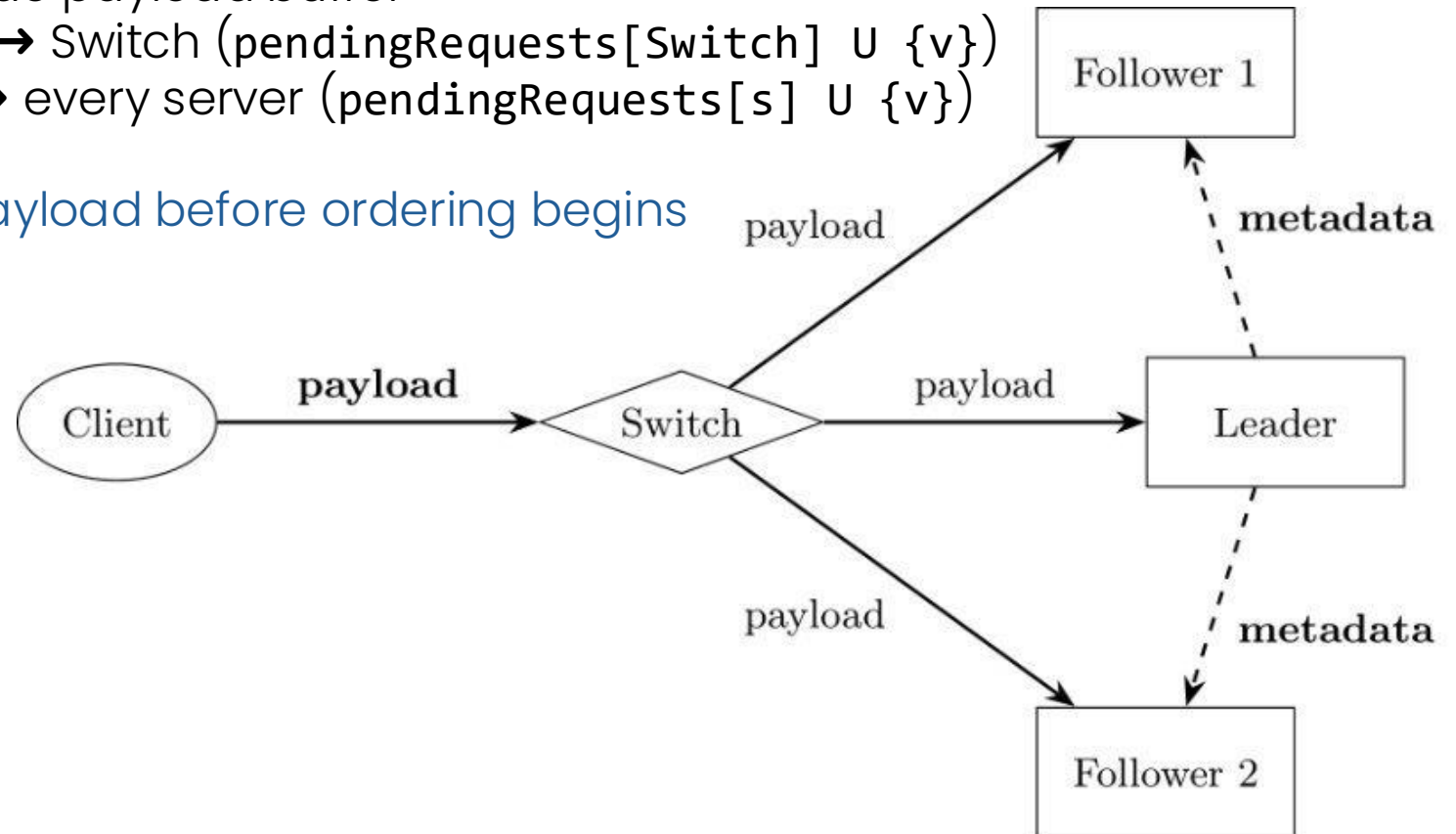
1. Switch component

- Receives client payloads once
- Multicasts payloads to *all* Raft servers

2. TLA+ variables / actions

- **CONSTANTS** Switch – distinct entity
- `pendingRequests[node]` – per-node payload buffer
- `SwitchClientRequest(v)` – client $\rightarrow$ Switch (`pendingRequests[Switch] U {v}`)
- `SwitchDisseminate(v)` – Switch $\rightarrow$ every server (`pendingRequests[s] U {v}`)

Outcome: All servers possess the raw payload before ordering begins



❖ HOVERCRAFT DESIGN – LEADER ORDERING & FOLLOWER RECONSTRUCTION

3. LeaderOrderRequest(i, v)
 - Leader selects v from pendingRequests[i]
 - Appends full entry [term, value=v, payload=v] to log[i]
4. AppendEntries(i, j)
 - Sends *metadata only* [term, value] to follower j
5. HandleAppendEntriesRequest(i, j, m) (*follower side*)
 - Looks up v in pendingRequests[i]
 - Reconstructs full entry; appends to log[i]
 - If payload missing → initiates *RecoveryRequest*

Benefit: Leader's replication path carries minimal data; payload never resent



❖ **LIVENESS IN HOVERCRAFT – FUNDAMENTALS & KEY STAGES**

What is Liveness?

- “Something good eventually happens.”
- Client requests are eventually processed and committed.
- Depends on a stable leader (standard Raft liveness).

Key Hovercraft Stages

1. Payload Dissemination (Switch → Servers)
 - Will every non-faulty server's `pendingRequests` cache receive the payload?
 - Requires `SwitchClientRequest` & `SwitchDisseminate` actions to eventually fire (fair Switch scheduling).
2. Leader Ordering (Leader Log)
 - Will a payload in the leader's `pendingRequests` be ordered and logged?
 - Requires `LeaderOrderRequest` to be fairly scheduled under a stable leader.

❖ LIVENESS – REPLICATION, RECONSTRUCTION & RECOVERY

```
EventuallyClientRequestProcessed(v) ==  
  <> ∃ i ∈ Server, idx ∈ DOMAIN log[i] :  
    log[i][idx].value = v ∧ commitIndex[i] ≥ idx  
  
EventuallyLeaderOrdersPendingRequest(leader, v) ==  
  (v ∈ pendingRequests[leader]) ~>  
  <> ∃ idx ∈ DOMAIN log[leader] : log[leader][idx].value = v
```

Continued Stages:

3. Metadata Replication (Leader → Followers)

- Will [term, value-ID] references reach a quorum via AppendEntries?
- Relies on Raft's retry logic and eventual message delivery.

4. Follower Log Construction

- With payload in cache + metadata from leader, will follower reconstruct and append [term, value, payload]?
- Requires HandleAppendEntriesRequest to run fairly.

5. Payload Recovery (*loss-mitigation*)

- If initial multicast is missed, will RecoveryRequest/Response eventually deliver the payload?
- Depends on leader availability and reliable exchange of recovery messages.

❖ LIVENESS DISCUSSION & CONCLUSION

Liveness Dependencies:

Standard Raft: leader stability, reliable server-to-server links, live processes.

Hovercraft Additions:

- Switch liveness (process SwitchClientRequest / SwitchDisseminate).
- Reliable Switch → Server payload delivery (mitigated by recovery loop).

Resource bounds: pendingRequests limited by MaxClientRequests.

(Optional) Hovercraft++ Impacts

- Batching metadata boosts throughput; does not alter core liveness assumptions.
- Leaderless/optimistic reads follow a distinct progress path.

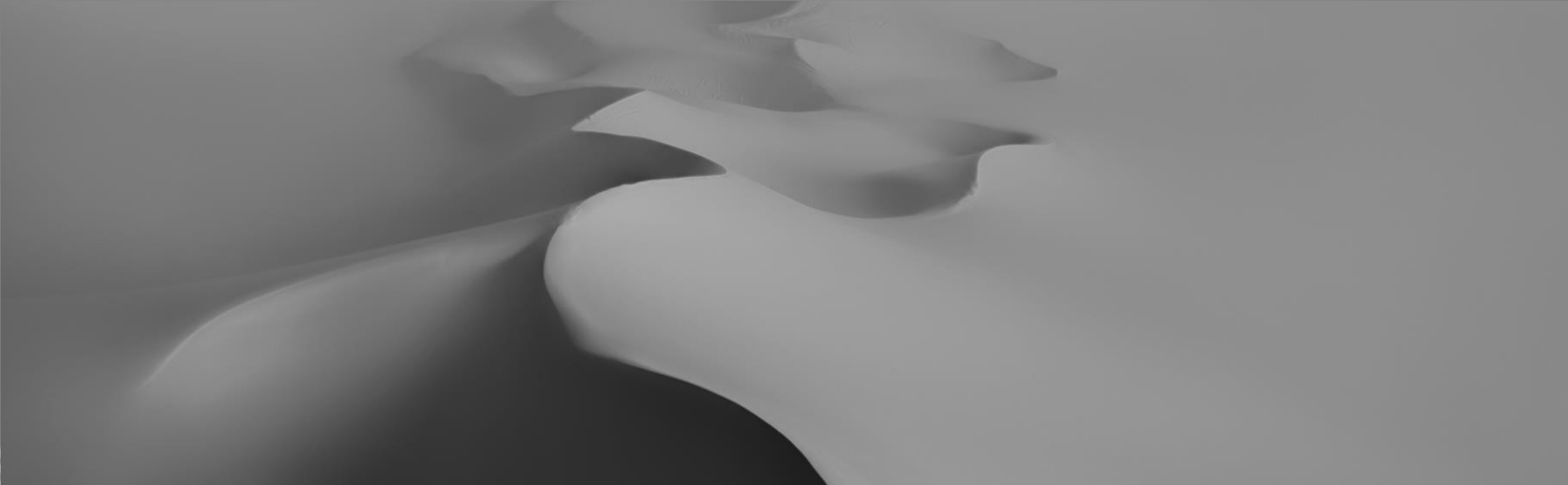
Conclusion:

Safety: TLA+ invariants (LogInv, LeaderCompletenessInv, ...) upheld; payload-metadata separation preserves Raft guarantees.

Liveness: End-to-end progress hinges on:

1. Switch disseminates payload.
2. Stable leader orders entry.
3. Metadata replicates to quorum.
4. Followers reconstruct entry (or recover payload).

Key Takeaway: Decoupling large payloads from ordering improves scalability while retaining Raft-style safety and progress—provided the Switch and recovery mechanism remain live.



THANK YOU

Presented by Parsa VARES

